The Attention Mechanism

Deep Learning

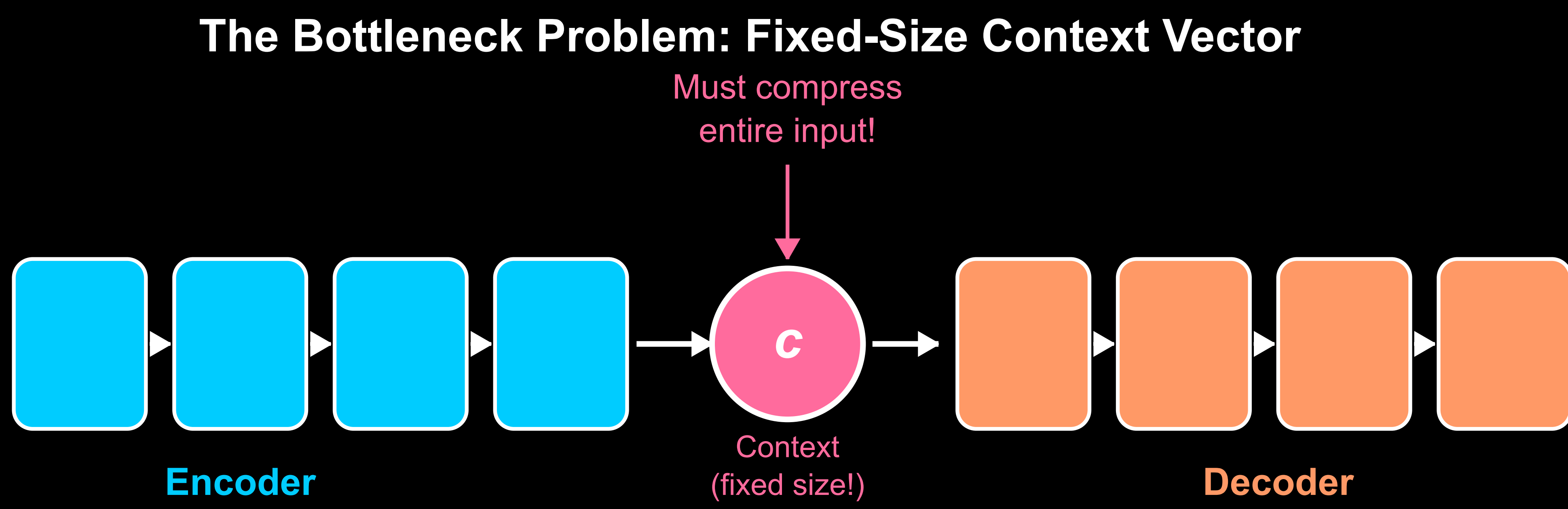
Daniel E. Acuna
Associate Professor, University of Colorado Boulder

Contents of This Video

In this video, we will cover:

- The bottleneck problem in encoder-decoder models
- The attention intuition
- Computing attention scores
- Attention weights and context vector
- How attention improves translation

The Encoder-Decoder Bottleneck



Problem: Entire input sentence must fit in one vector!

The Attention Intuition

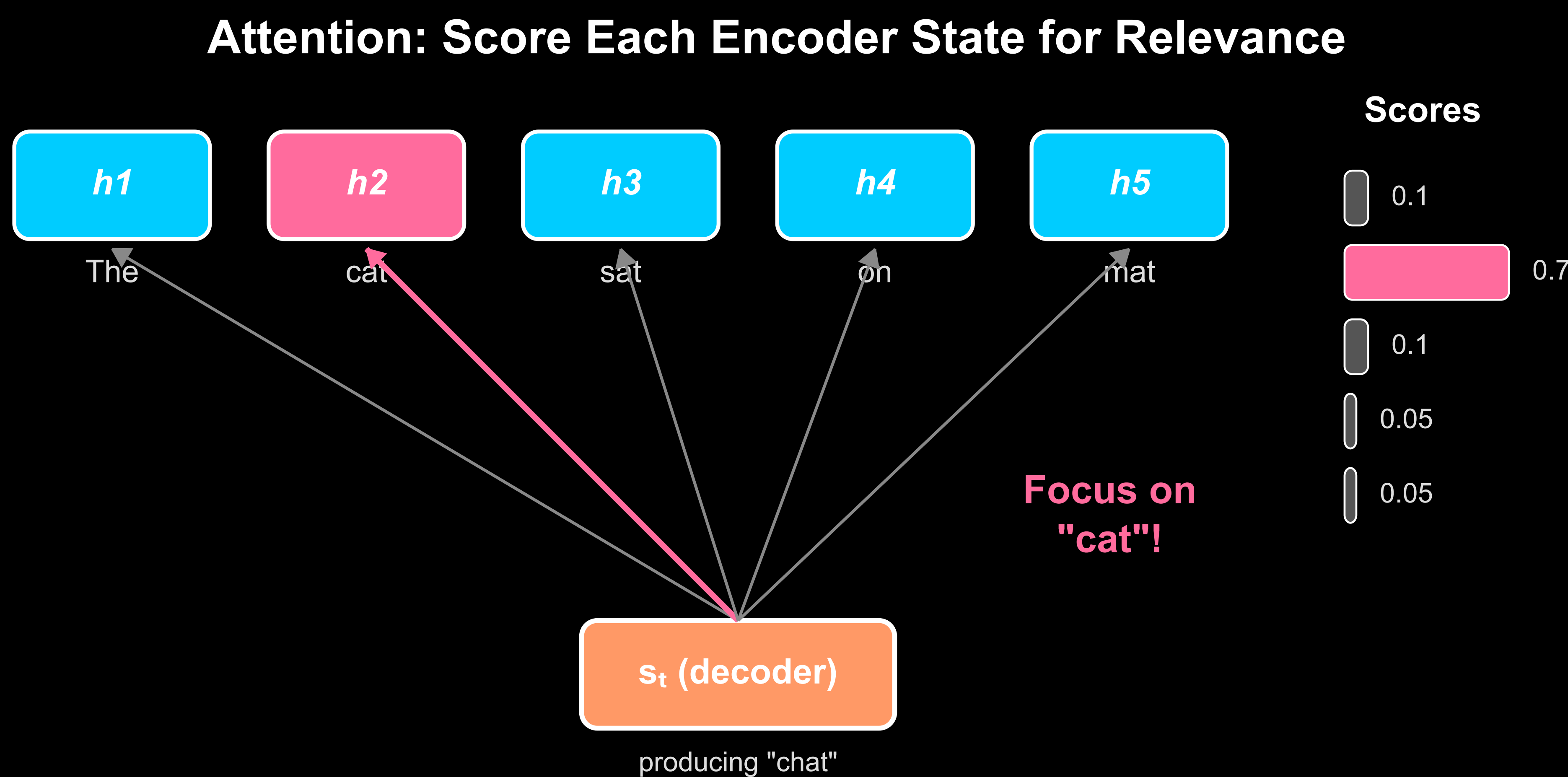
Human analogy: When translating, you look back at specific source words

“The cat sat on the mat” → **“Le chat était assis sur le tapis”**

- When producing “chat” → focus on “cat”
- When producing “assis” → focus on “sat”
- When producing “tapis” → focus on “mat”

Attention: Let decoder “look at” all encoder outputs, focusing on relevant ones

Computing Attention



Attention Weights

Score function:

$$e_{ti} = \text{score}(s_t, h_i)$$

Normalize with softmax:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_j \exp(e_{tj})}$$

Properties of α:

- $\alpha_{ti} \geq 0$
- $\sum_i \alpha_{ti} = 1$

Interpretation: Probability distribution over input positions

Context Vector

Weighted sum of encoder states:

$$c_t = \sum_i \alpha_{ti} h_i$$

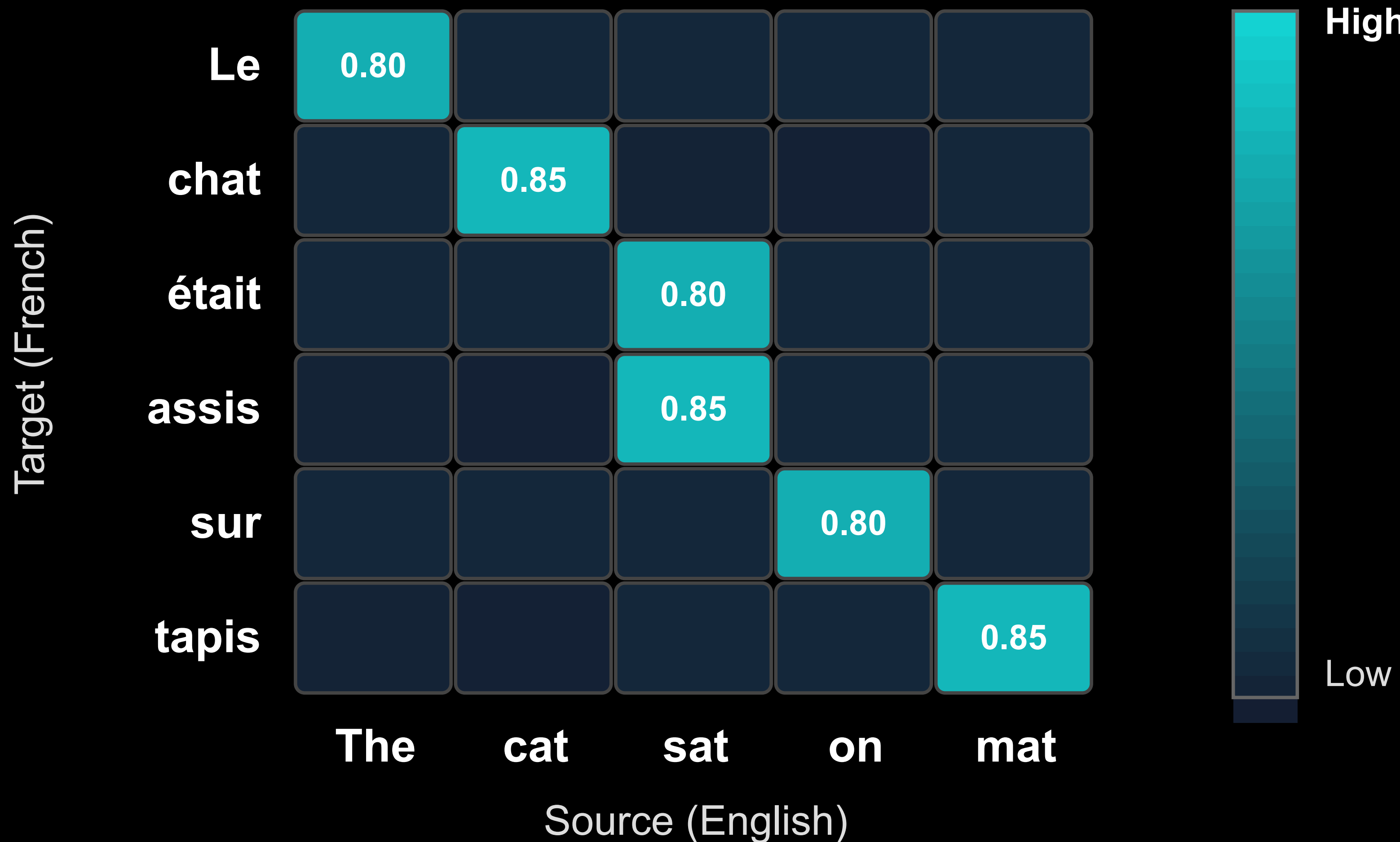
This context vector:

- Is different for each decoder timestep t
- Focuses on relevant encoder positions
- Is fed to the decoder along with previous output

No more fixed bottleneck!

Attention Visualization

Attention Matrix: Decoder Focuses on Relevant Source Words



What We've Covered

In this video, we've learned:

- Encoder-decoder bottleneck: fixed-size context loses information
- Attention: decoder "looks at" all encoder states
- Scores measure relevance between decoder and encoder states
- Softmax converts scores to attention weights
- Context vector = weighted sum of encoder states
- Different context for each decoder timestep (no bottleneck!)